



Government  
of Canada

Gouvernement  
du Canada

## **ESDC Development Community**

# TFS to GCcode (or GitHub)

*Le texte français est donné à la suite.*

### **Similar Guides:**

- [TFVC to Git](#)

## **Steps**

1. *(Optional)* Decouple projects and services
2. Scan & Remove secrets (encrypt where necessary)
3. Migrate TFS Tasks to Issues (manual, overtime, after a release)
4. Migrate the Code
5. Build your pipeline

## Setting up a GCcode repository for your team

1. Create a personal account in GCcode
2. Create an issue in the ESDC Welcome project and fill out the New Team Group Request template
3. Wait for access to your new group
4. Create a New project
5. Clone and copy the files from the template-gabarit project
6. Use the label-generator to add default starter labels to your project

*Note: You may need to configure your repository to bypass the network proxy to be able to clone from GitHub.*

## Setting up a GitHub repository for your team

1. Create a personal account in [GitHub](#)
2. Create a new Organization for your team
3. Create an issue in the [ESDC Welcome](#) project and fill out the List Team Group template
4. Fork the [template-gabarit](#) project into your new organization.
5. Change the *Settings* of that forked project to be a *Template repository* (under *Repository name*)
6. Create a new repository using the forked template project as the template
7. Use the [label-generator](#) to add default starter labels to your project

*Note: You may need to configure your repository to bypass the network proxy to be able to clone from GitHub.*

## FAQ

### How do you manage TFS related items like UserStory, WorkItem, Bugs, etc, in GitLab?

It looks very similar in GitLab.

### Labels & Issue Templates!!!

*Labels* are critical to organizing *Issues* (or *WorkItems*). *Bugs* are given a bug label. Things like *UserStory* are given an enhancement or feature request label.

*Issue Templates* help users creating new *Issues* to fill out all of the necessary details for a standard *Issue*. You can copy bug and feature *Issue templates* from the [ESDC Template](#) project.

We recommend that you start with the *key* labels from [ESDC Label Generator](#) project to add default labels to your project or group and add more as it helps you organize your project. You should have labels that identify key parts of your application. You should also add labels to help with the flow of your team's processes.

### **Is it possible to migrate all the Work Items (User Story, Bug, tasks, etc) to GitLab?**

Even if it's look very different.

It might be technically possible by custom writing a script using TFS and GitLab APIs; but I wouldn't recommend it. The platforms are different enough that it could take more time to write the script than doing it manually. The other benefit of doing it manually is forcing your team to remove redundant and outdated information. (Don't move closed items.)

### **What is the recommended approach for managing secrets in code?**

When migrating to an open source platform (like GitHub or GCcode) we can't keep hardcoding secrets, and they need to be removed. How should a new developer get the necessary information to input into the web.config? (i.e. : inserting secrets, server name, username, etc)

Whatever tool is used to building your solution for deployment is also responsible for injecting the secrets into the config. Depending on the language and framework you are developing with, they may have different methods to managing the injection of those secrets. You may need to custom write a build script to search and replace secrets from your build's secret environment variables. GitLab, Jenkins, Azure DevOps & TFS 2015 all have ways of managing secret environment variables for your scripted build.

A new developer should likely be creating their own development secrets. As for non-user specific secrets, it likely will depend on the secret itself as to how the developer will obtain that information.

## **What is the recommended approach for working with an external testing team service?**

When migrating on GCcode, all the functionality around the work items changes. In GcCode, everything is related around `issues` combined with different `labels`. With TFS, we have some strict workflow where work items have strict state flow. Testing team are working with tools like 'Microsoft Test Manager' which is tightly coupled with TFS.

Get them to work with you. Create workflows that work with your development cycle; there is nothing stopping a team from having strict workflows and following them in GCcode.

As far as the tool usage goes, developers and testers need to pick the tools that are best for the project. GCcode does function differently than TFS, however GCcode is more mature and flexible as a platform for

development. If GCcode is the development platform (which it has been recommended to be) the testing platform (or tools) need to work with it, not the other way around.

---

*Texte français:*

## **Guides similaires :**

- [De TFVC à Git](#)

## **Étapes**

1. *(Optionnel)* Découpler les projets et les services
2. Analyser et retirer les secrets (chiffrer lorsque nécessaire)
3. Migrer les tâches TFS vers des tickets (Issues) (manuellement, au fil du temps, après une livraison)
4. Migrer le code
5. Bâtir votre pipeline

## **Configurer un référentiel GCcode pour votre équipe**

1. Créez un compte personnel sur [GCcode](#)
2. Créez un ticket dans le projet [ESDC Welcome](#) et remplissez le modèle New Team Group Request
3. Attendez d'obtenir l'accès à votre nouveau groupe
4. Créez un nouveau projet ( New project )
5. Clonez et copiez les fichiers du projet [template-gabarit](#)
6. Utilisez le [générateur d'étiquettes](#) pour ajouter les étiquettes de démarrage par défaut à votre projet

*Remarque : Vous pourriez devoir configurer votre référentiel pour contourner le proxy réseau afin de pouvoir cloner depuis GitHub.*

## Configurer un référentiel GitHub pour votre équipe

1. Créez un compte personnel sur [GitHub](#)
2. Créez une nouvelle organisation pour votre équipe
3. Créez un ticket dans le projet [ESDC Welcome](#) et remplissez le modèle List Team Group
4. Bifurquez (fork) le projet [template-gabarit](#) dans votre nouvelle organisation
5. Modifiez les paramètres (*Settings*) de ce projet forké pour en faire un référentiel modèle (*Template repository*)
6. Créez un nouveau référentiel en utilisant ce modèle
7. Utilisez le [générateur d'étiquettes](#) pour ajouter les étiquettes de démarrage par défaut à votre projet

*Remarque : Vous pourriez devoir configurer votre référentiel pour contourner le proxy réseau afin de pouvoir cloner depuis GitHub.*

## FAQ

### Comment gérer les éléments de type UserStory, WorkItem, bogues dans GitLab ?

C'est très similaire dans GitLab.

### Étiquettes et modèles de tickets (Labels & Issue Templates) !!!

Les *étiquettes* sont essentielles pour organiser les *tickets* (ou *éléments de travail*). Les anomalies reçoivent l'étiquette `bug`. Les éléments de type récit utilisateur reçoivent l'étiquette `enhancement` ou `feature request`.

Les *modèles de tickets* aident les utilisateurs à fournir tous les détails nécessaires lors de la création d'un ticket standard. Vous pouvez copier les modèles de tickets `bug` et `feature` à partir du projet de modèle d'EDSC.

Nous recommandons de commencer par les étiquettes de base du générateur d'étiquettes d'EDSC pour votre groupe ou projet, puis d'en ajouter au besoin pour identifier les composants clés et fluidifier vos processus d'équipe.

### Est-il possible de migrer tous les éléments de travail (User Stories, bogues, tâches) vers GitLab ?

Même si la structure est différente.

C'est techniquement réalisable en développant un script sur mesure exploitant les API de TFS et de GitLab, mais nous ne le conseillons pas. Les plateformes présentent des divergences telles que la rédaction du script peut s'avérer plus longue qu'un traitement manuel. Le traitement manuel permet en outre de faire le tri et d'éliminer les informations désuètes. (Ne migrez pas les éléments fermés.)

## Quelle est l'approche recommandée pour la gestion des secrets dans le code ?

Lors de la migration vers une plateforme ouverte (comme GitHub ou GCcode), nous ne pouvons plus conserver de secrets codés en dur. Comment un nouveau développeur doit-il obtenir ces informations (serveurs, mots de passe, etc.) ?

L'outil utilisé pour compiler et déployer la solution est également responsable de l'injection des secrets dans la configuration. Selon le langage et le cadriciel utilisés, différentes méthodes permettent d'injecter ces valeurs via les variables d'environnement sécurisées. GitLab, Jenkins, Azure DevOps et TFS 2015 disposent tous de mécanismes de gestion des variables secrètes pour vos scripts de compilation.

Un nouveau développeur devrait généralement générer ses propres secrets de développement local. Quant aux secrets non individuels, leur obtention dépendra de la politique de sécurité propre au composant.

## Quelle démarche adopter pour collaborer avec une équipe de test externe ?

Sur GCcode, la gestion des éléments repose sur les tickets et les étiquettes, alors que TFS applique des flux d'état très rigides et s'intègre étroitement avec Microsoft Test Manager.

Invitez-les à collaborer avec votre équipe. Établissez des flux de travail adaptés à votre cycle de développement; rien n'empêche de formaliser des flux stricts et de les suivre avec rigueur dans GCcode.

Les développeurs et testeurs doivent choisir les outils les plus performants pour le projet. GCcode offre une flexibilité et une maturité supérieures pour le développement. Dès lors que GCcode sert de plateforme de développement (ce qui est recommandé), les outils de test doivent s'y intégrer harmonieusement.